

Reviewer Pack — Minimal Geometric Entropy and DPLL Proof Width

Entry a70a75ba44cb · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-05 12:20:15 UTC

Minimal Geometric Entropy and DPLL Proof Width

Entry ID: a70a75ba44cb **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-05 12:20:15 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Geometric Entropy) **Field B** (complexity object): Complexity Theory (DPLL Proof Complexity)

Statement:

For every Boolean satisfiability problem φ , the geometric entropy H_φ of its associated cube complex is linearly correlated with its DPLL proof width $w_{\text{DPLL}}(\varphi)$, such that $H_\varphi = \Theta(w_{\text{DPLL}}(\varphi))$.

Rationale (proposer's reasoning):

Geometric entropy measures the complexity of a space in terms of its fractal dimension and could potentially reflect the depth of computation required to solve a problem. If this correlation holds, it suggests a deep connection between geometric properties of problems and their computational hardness.

Taxonomy category: `minimal_geometric_entropy_dpll` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): efa651c660dfd1bc

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for at least 95% of the generated CNF formulas with n variables, the correlation coefficient between geometric entropy H_φ and DPLL proof width $w_{\text{DPLL}}(\varphi)$ is greater than or equal to 0.8, and there is no seed that results in a correlation coefficient less than 0.5.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "geometric entropy" AND "DPLL proof complexity" - "cube complex" IN ALGTOP AND "Boolean satisfiability problem" IN COMPLEXITY-THEORY - " $H_\phi = \theta(w_{DPLL}(\phi))$ "

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=241.0s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_cnf(n):
        clauses = []
        for _ in range(2**n):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
            if all(x == 0 for x in clause):
                continue
            clauses.append(clause)
        return clauses

    def dpll(cnf, assignment={}):
        if not cnf:
            return True
        var = next((v for v in range(1, len(assignment) + 2) if v not in assignment), None)
        if var is None:
            return False

        pos_var = var
        neg_var = -var
        pos_clauses = [c for c in cnf if any(x == pos_var for x in c)]
        neg_clauses = [c for c in cnf if any(x == neg_var for x in c)]

        if dpll(pos_clauses, assignment | {pos_var: True}):
            return True
        elif dpll(neg_clauses, assignment | {neg_var: False}):
            return True

        return False
```

```

def geometric_entropy(cnf):
    n = len(cnf[0])
    box_count = 0
    scale = 1.0
    while True:
        count = 0
        for i in range(-int(scale), int(scale) + 1):
            for j in range(-int(scale), int(scale) + 1):
                if all(x * (i + j) <= 0 for x in cnf[0]):
                    count += 1
        box_count += count
        scale *= 2
        if scale > 100:
            break
    return math.log(box_count, 2)

n_values = [5, 10, 15, 20, 30, 40]
results = []
for n in n_values:
    cnf = generate_cnf(n)
    entropy = geometric_entropy(cnf)
    width = dpll(cnf)
    results.append((n, entropy, width))

if len(results) < 30:
    return {
        "metric_name": "correlation_coefficient",
        "metric_value": None,
        "instances_tested": len(results),
        "n_max": max(n for n, _, _ in results),
        "conjecture_holds": False,
        "counterexample": "not_enough_instances"
    }

entropies = [r[1] for r in results]
widths = [r[2] for r in results]
correlation_coefficient = sum((e - sum(entropies) / len(entropies)) * (w - sum(widths) / len(w
    return {
        "metric_name": "correlation_coefficient",
        "metric_value": correlation_coefficient,
        "instances_tested": len(results),
        "n_max": max(n for n, _, _ in results),
        "conjecture_holds": correlation_coefficient >= 0.8 and all(corr >= 0.5 for corr in [correl
        "counterexample": "" if correlation_coefficient >= 0.8 else "low_correlation"
    }

if __name__ == "__main__":
    import sys
    seeds = list(map(int, sys.argv[1:])) or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

```

```

mean_value = sum(r["metric_value"] for r in results if r["metric_value"] is not None) / len(re
std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results if r["metric_va
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fra
elif any(not r["conjecture_holds"] and r["metric_value"] < 0.5 for r in results):
    first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["con
    print(f"RESULT: FALSIFIED counterexample='low_correlation' first_failing_seed={first_faili
else:
    print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction} n_tested={len(results)}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not meet the pre-registered support condition for the conjecture. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14330
2	propose	ollama_remote	glm4:latest	0	0	12491
3	preregistration	ollama_remote	glm4:latest	0	0	9523
4	novelty	ollama_remote	glm4:latest	0	0	11669
5	novelty	ollama_remote	glm4:latest	0	0	9052
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	22344
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14491
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12850
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	43511
10	judge	ollama_remote	glm4:latest	0	0	62153

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 212414 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/a70a75ba44cb.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/a70a75ba44cb.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/a70a75ba44cb.tar.gz` (if generated)